



PyChem Brief Manual

A Pure-Python Desktop Application and Library for Chemistry and Cheminformatics

Table of Contents

1. Introduction
2. Installation Guide
3. System Requirements
4. Quick Start
5. Core Functions and Applications
6. Selection Algebra
7. Plugin System
8. Python API Reference
9. GUI Features
10. Advanced Usage
11. Troubleshooting
12. Development Guide

Introduction

PyChem is a pure-Python desktop application and library for chemistry and cheminformatics. It provides molecular visualization, SMILES parsing, PDB loading, MMFF94 geometry optimization, molecular descriptors, and a plugin ecosystem - all implemented from scratch without external chemistry libraries like RDKit or OpenBabel.

Key Features

- **Pure Python:** No C++ extensions, completely readable implementation
- **Educational:** Perfect for academic use and learning computational chemistry
- **Portable:** Runs identically on Windows, macOS, and Linux
- **Extensible:** Service-oriented architecture with plugin system
- **Comprehensive:** 2D/3D visualization, force field optimization, descriptors

Architecture Overview

PyChem follows a layered architecture:

...

Public API Layer (pychem/) - No Qt dependency, Jupyter-friendly

|
Service Registry - Protocol-based service interfaces

|

Core Services - ForceField, Renderer, Loader, Descriptors, etc.

|
Core Domain - Molecule, Atom, Bond, Element models

|
Infrastructure - EventBus, ParallelExecutor, Security
```

---

## Installation Guide:

### Prerequisites

- **Python:** 3.10 or newer (3.13/3.14 supported)
- **Operating System:** Windows 10/11, macOS 12+, or Linux with Qt 6 support
- **RAM:** 4 GB minimum, 8 GB recommended for large proteins
- **CPU:** Multi-core recommended for parallel processing

### ### Step 1: Clone the Repository

```
```bash
git clone https://github.com/vijaymasand/PyChem.git
cd PyChem
```
```

### ### Step 2: Create Virtual Environment

#### Windows (Command Prompt):

```
```bash
python -m venv venv
venv\Scripts\activate.bat
```
```

#### Windows (PowerShell):

```
```bash
python -m venv venv
venv\Scripts\Activate.ps1
```
```

#### macOS/Linux:

```
```bash
python3 -m venv venv
source venv/bin/activate
```
```

### ### Step 3: Install Dependencies

```
```bash
pip install --upgrade pip
```

```
pip install -r requirements.txt
```
```

This installs:

- `PySide6 >= 6.5.0` (Qt GUI framework)
- `numpy >= 1.24.0` (Numerical computing)
- `matplotlib >= 3.7` (Plotting - optional)
- `pandas >= 2.0` (Data analysis - optional)
- `psutil >= 5.9` (System utilities)
- `nuitka >= 2.0` (For creating standalone binaries)
- `pillow >= 10.0` (Image processing)

### ### Step 4: Install Plugin Dependencies (Optional)

For full plugin functionality, install additional dependencies:

```
```bash
pip install packaging pandas scipy scikit-learn matplotlib rdkit
```
```

### ### Step 5: Verify Installation

```
```bash
python main.py
```
```

The application should launch with the PyChem GUI. First launch generates a 10-year development license automatically.

### ### Building Standalone Binary (Optional)

```
```bash
python build.py
```
```

This creates a compiled executable in the `build/` directory using Nuitka.

---

## ## System Requirements

### ### Minimum Requirements

- **OS:** Windows 10/11, macOS 12+, Ubuntu 22.04+, Fedora 37+
- **Python:** 3.10 or newer
- **RAM:** 4 GB
- **Storage:** 500 MB free space
- **Display:** 1024x768 resolution

### ### Recommended Requirements

- **OS:** Windows 11, macOS 13+, Ubuntu 24.04+
- **Python:** 3.11 or newer
- **RAM:** 8 GB (for proteins >5000 atoms)
- **Storage:** 2 GB free space
- **Display:** 1920x1080 resolution
- **GPU:** OpenGL 3.3+ support (optional, for future hardware acceleration)

### ### Cross-Platform Compatibility

PyChem is designed to work identically across platforms:

- **Multiprocessing:** Uses `spawn` start method for cross-platform safety
- **File Paths:** Uses `pathlib` for cross-platform path handling
- **Fonts:** Fallback font system for missing platform-specific fonts

---

## ## Quick Start

### ### Launching the Application

```
```bash
# From the project root
python main.py
```
```

### ### Basic GUI Usage

1. **Load a Molecule:** File > Open > Select PDB/MOL/SDF file
2. **SMILES Input:** Enter SMILES string in the input field and press Enter
3. **3D Optimization:** Chemistry > Optimize Geometry
4. **View Modes:** Toggle between ball-and-stick, space-fill, cartoon, etc.
5. **Export:** File > Export Image or File > Save As

### ### Python API Quick Start

```
```python
import pychem

# Parse SMILES
mol = pychem.parse_smiles("CCO") # ethanol
print(f"Molecular formula: {mol.molecular_formula()}") # C2H6O

# Generate 3D coordinates and optimize
pychem.generate_3d(mol)
result = pychem.optimize(mol, max_iters=500)
```

```

print(f"Optimized energy: {result.final_energy:.2f} kcal/mol")

# Calculate descriptors
desc = pychem.descriptors(mol)
print(f"Molecular weight: {desc['molecular_weight']:.2f}")

# Load a protein
protein = pychem.load("protein.pdb")
print(f"Protein has {protein.num_atoms} atoms")
```

```

## ## Core Functions and Applications

### ### Chemistry Functions

#### #### SMILES Parsing

```

```python
# Parse various SMILES formats
mol = pychem.parse_smiles("CCO") # Simple organic
mol = pychem.parse_smiles("c1ccccc1") # Aromatic
mol = pychem.parse_smiles("[NH4+]") # Charged species
mol = pychem.parse_smiles("C[C@H](O)C") # Stereochemistry
```

```

#### #### File Loading

```

```python
# Load different file formats
mol = pychem.load("molecule.pdb") # Protein structure
mol = pychem.load("compound.mol") # MDL Molfile
mol = pychem.load("library.sdf") # Structure Data File
mol = pychem.load("complex.mol2") # Tripos Mol2
```

```

#### #### 3D Coordinate Generation

```

```python
# Generate 3D coordinates with optimization
pychem.generate_3d(mol, optimize=True, max_steps=200)

# Generate coordinates without optimization
pychem.generate_3d(mol, optimize=False)
```

```

#### #### MMFF94 Force Field

```

```python
# Full geometry optimization
result = pychem.optimize(mol, max_iters=500, method='lbfgs')
```

```

```

print(f"Converged: {result.converged}")
print(f"Final energy: {result.final_energy:.2f} kcal/mol")
print(f"Optimization steps: {result.num_steps}")

Alternative optimization method
result = pychem.optimize(mol, method='steepest_descent')
...

Partial Charges
```python
# Add hydrogens and assign charges
h_count = pychem.add_hydrogens(mol)
print(f"Added {h_count} hydrogens")

# Assign MMFF94 BCI charges only
pychem.compute_charges(mol)

# View charges
for atom in mol.atoms:
    print(f"{atom.symbol}{atom.index}: {atom.partial_charge:+.4f}")
...

### Molecular Descriptors

#### Basic Descriptors
```python
desc = pychem.descriptors(mol)
print(f"Molecular weight: {desc['molecular_weight']:.2f}")
print(f"Number of atoms: {desc['num_atoms']}")
print(f"Number of heavy atoms: {desc['num_heavy_atoms']}")
print(f"Number of bonds: {desc['num_bonds']}")
print(f"Number of rings: {desc['num_rings']}")
print(f"Total charge: {desc['total_charge']}")
...

Specific Descriptor Categories
```python
# Constitutional descriptors
constitutional = pychem.descriptors(mol, names=[
    'molecular_weight', 'num_atoms', 'num_heavy_atoms',
    'num_bonds', 'num_rings', 'formula'
])

# Topological descriptors
topological = pychem.descriptors(mol, names=[
    'wiener_index', 'randic_index', 'balaban_index',
    'harary_index', 'hyper_wiener_index'
])

```

```

# Electronic descriptors
electronic = pychem.descriptors(mol, names=[
    'dipole_moment', 'homo_lumo_gap', 'electronegativity',
    'hardness', 'softness', 'electrophilicity'
])
...

#### Batch Processing
```python
Process multiple molecules in parallel
smiles_list = ["CCO", "c1ccccc1", "CC(=O)O", "CCCCC"]
molecules = [pychem.parse_smiles(s) for s in smiles_list]
batch_results = pychem.descriptors_batch(molecules)

for i, desc in enumerate(batch_results):
 print(f"{smiles_list[i]}: MW = {desc['molecular_weight']:.2f}")
...

```

### ### Visualization Functions

#### #### 2D Structure Display

- Automatic 2D coordinate generation
- Bond rendering (single, double, triple, aromatic)
- Wedge/hash stereochemical bonds
- Atom labels and formal charges
- Selection highlighting

#### #### 3D Molecular Visualization

- Ball-and-stick, space-fill, wireframe modes
- Protein cartoon rendering (alpha-helix, beta-sheet, coil)
- Color schemes: by element, secondary structure, rainbow, chain
- High-DPI image export (72-600 DPI)
- Ray-traced publication-quality images

---

### ## Selection Algebra

PyChem provides a powerful selection system for working with molecular structures. The selection algebra allows you to specify atoms and groups of atoms using natural language-like expressions.

### ### Basic Selection Syntax

#### #### Element Selection

```

```python
# Select specific elements
sele('C')          # All carbon atoms

```

```

sele('N')          # All nitrogen atoms
sele('O')          # All oxygen atoms
sele('H')          # All hydrogen atoms
'''

```

Property Selection

```

'''python
# Select by atomic properties
sele('organic')    # All non-hydrogen atoms in organic molecules
sele('backbone')   # Protein backbone atoms
sele('sidechain')  # Protein side chain atoms
'''

```

Spatial Selection

```

'''python
# Select by spatial relationships
sele('within 5.0 COM')    # Atoms within 5.0 Å of center of mass
sele('within 3.5 of chain A') # Atoms near chain A
sele('farther 10.0 from COM') # Atoms farther than 10.0 Å from COM
'''

```

Structure Selection

```

'''python
# Select by structural features
sele('ring')          # Atoms in rings
sele('aromatic')      # Atoms in aromatic systems
sele('helix')         # Atoms in alpha helices
sele('sheet')         # Atoms in beta sheets
'''

```

Chain and Residue Selection

```

'''python
# Protein-specific selections
sele('chain A')       # All atoms in chain A
sele('residue 50')    # Atoms in residue 50
sele('ALA')           # All alanine residues
sele('water')         # Water molecules
'''

```

Complex Selections

Boolean Operations

```

'''python
# Combine selections with AND, OR, NOT
sele('C and within 3.0 of N')    # Carbon atoms near nitrogen
sele('helix or sheet')           # Secondary structure atoms
sele('not H')                    # All non-hydrogen atoms
sele('(C or N) and not aromatic') # Non-aromatic C or N atoms
'''

```



```
...
```

Nested Selections

```
```python
Complex nested expressions
sele('chain A and (helix or sheet)') # Secondary structure in chain A
sele('within 5.0 of (ALA or GLY)') # Near specific residues
sele('organic and not (within 2.0 of water)') # Organic atoms away from water
```
```

Selection in Python Console

```
```python
In the Python console (mol is available)
selected_atoms = sele('C and within 3.0 of N')
print(f"Selected {len(selected_atoms)} atoms")

Apply selection to visualization
viewer_3d.set_selected(selected_atoms)
viewer_2d.set_selected(selected_atoms)

Calculate properties for selection
for atom_idx in selected_atoms:
 atom = mol.atoms[atom_idx]
 print(f"Atom {atom_idx}: {atom.symbol}, charge = {atom.partial_charge}")
```
```

Selection Examples

```
```python
Common selection patterns
sele('protein') # All protein atoms (not water/ligands)
sele('ligand') # All non-protein atoms
sele('active_site') # Atoms within 6.0 Å of ligand
sele('hydrophobic') # Non-polar side chain atoms
sele('polar') # Polar side chain atoms
sele('charged') # Charged atoms
sele('metal') # Metal atoms
```
```

```
---
```

Plugin System

PyChem features a comprehensive plugin system that allows users to extend functionality with custom analysis, visualization, and I/O capabilities.

Plugin Architecture

Base Plugin Class

All plugins inherit from `BasePlugin` and implement required methods:

```
```python
from src.plugins.base_plugin import BasePlugin, PluginWidget
from src.plugins.plugin_types import PluginInfo, PluginType

class MyPlugin(BasePlugin):
 def __init__(self):
 super().__init__(
 info=PluginInfo(
 name="My Plugin",
 version="1.0.0",
 description="Custom analysis plugin",
 author="Your Name",
 plugin_type=PluginType.ANALYSIS,
 dependencies=["numpy", "matplotlib"]
)
)

 def create_widget(self):
 """Create the plugin's main widget."""
 return MyPluginWidget(self)

 def initialize(self, main_window, api):
 """Initialize plugin with application access."""
 self.main_window = main_window
 self.api = api
 return True

 def cleanup(self):
 """Clean up plugin resources."""
 pass

 def on_molecule_changed(self, molecule):
 """Handle molecule changes."""
 if molecule:
 self.update_analysis(molecule)
...
```
```

Plugin Widget Class

```
```python
class MyPluginWidget(PluginWidget):
 def __init__(self, plugin):
 super().__init__(plugin)
 self.setup_ui()
```
```

```

def setup_ui(self):
    """Setup the plugin's user interface."""
    from src.shared.qt_compat import QWidget, QVBoxLayout, QLabel

    self.widget = QWidget()
    layout = QVBoxLayout(self.widget)

    self.label = QLabel("No molecule loaded")
    layout.addWidget(self.label)

def on_molecule_changed(self, molecule):
    """Update display when molecule changes."""
    if molecule:
        self.label.setText(f"Analyzing {molecule.num_atoms} atoms")
    else:
        self.label.setText("No molecule loaded")
...

### Plugin Types

#### Analysis Plugins
Perform calculations and data analysis:

```python
class AnalysisPlugin(BasePlugin):
 """Example: Molecular weight calculator"""

 def create_widget(self):
 return WeightCalculatorWidget(self)

 def calculate_properties(self, molecule):
 """Calculate molecular properties."""
 if not molecule:
 return {}

 properties = {
 'molecular_weight': sum(atom.atomic_mass for atom in
molecule.atoms),
 'num_atoms': molecule.num_atoms,
 'num_heavy_atoms': len([a for a in molecule.atoms if a.symbol !=
'H']),
 'formula': molecule.molecular_formula()
 }
 return properties
...

Visualization Plugins
Create custom visualizations:

```

```

```python
class VisualizationPlugin(BasePlugin):
    """Example: Custom 2D structure viewer"""

    def create_widget(self):
        return Custom2DViewer(self)

    def setup_visualization(self, molecule):
        """Setup custom visualization."""
        # Create custom rendering
        pass
```

```

#### #### I/O Plugins

Add support for new file formats:

```

```python
class IOPlugin(BasePlugin):
    """Example: Custom file format reader"""

    def load_custom_format(self, filepath):
        """Load molecules from custom format."""
        molecules = []
        # Parse custom format
        return molecules

    def save_custom_format(self, molecules, filepath):
        """Save molecules to custom format."""
        # Write custom format
        pass
```

```

#### ### Built-in Plugins

##### #### Molecular Weight Calculator

Calculates molecular weight and element composition:

- Real-time weight calculation
- Element percentage composition
- Formula parsing and validation

##### #### Ramachandran Plot

Analyzes protein backbone conformation:

- Phi-psi angle plotting
- Outlier detection
- Secondary structure coloring

##### #### QSAR Modeler

Builds and validates quantitative structure-activity models:

- Multiple regression algorithms
- Cross-validation
- Model statistics and plots

#### #### Descriptor Pruning

Analyzes and reduces descriptor redundancy:

- Correlation analysis
- Variance filtering
- Feature importance ranking

#### #### Docking Pose Visualizer

Compares and analyzes docking poses:

- RMSD calculations
- Pose clustering
- Interaction analysis

### ### Plugin Development Guide

#### #### Step 1: Create Plugin File

Create a new Python file in the `plugins/` directory:

```
```python
# plugins/my_custom_plugin.py
from src.plugins.base_plugin import BasePlugin, PluginWidget
from src.plugins.plugin_types import PluginInfo, PluginType

class MyCustomPlugin(BasePlugin):
    def __init__(self):
        super().__init__(
            info=PluginInfo(
                name="My Custom Plugin",
                version="1.0.0",
                description="Custom functionality",
                author="Your Name",
                plugin_type=PluginType.ANALYSIS,
                dependencies=[]
            )
        )

    def create_widget(self):
        return MyCustomWidget(self)

class MyCustomWidget(PluginWidget):
    def __init__(self, plugin):
        super().__init__(plugin)
        self.setup_ui()
```

```

def setup_ui(self):
    # Setup your UI here
    pass
...

```

Step 2: Implement Required Methods

- `create_widget()`: Returns the plugin's main widget
- `initialize()`: Sets up plugin with application access
- `cleanup()`: Cleans up resources when unloaded
- `on_molecule_changed()`: Responds to molecule changes

Step 3: Use Plugin API

Access application functionality through the plugin API:

```

```python
def on_molecule_changed(self, molecule):
 if molecule:
 # Get current molecule
 mol = self.get_current_molecule()

 # Show status message
 self.show_status_message("Analyzing molecule...")

 # Update viewers
 self.update_viewer_2d()
 self.update_viewer_3d()

 # Add menu item
 self.add_menu_item("Tools.My Plugin", "Calculate", self.calculate)
...

```

#### #### Step 4: Plugin Installation

1. Place plugin file in `plugins/` directory
2. Restart PyChem
3. Plugin appears in Plugins menu
4. Or use Plugins > Install Plugin to load manually

#### ### Plugin API Reference

##### #### BasePlugin Methods

```

```python
# Core methods
create_widget()           # Required: Create main widget
initialize(main_window, api) # Optional: Initialize plugin
cleanup()                 # Optional: Clean up resources
on_molecule_changed(molecule) # Optional: Handle molecule changes
on_plugin_activated()      # Optional: Handle activation
on_plugin_deactivated()    # Optional: Handle deactivation

```

```

# API helper methods
get_current_molecule()      # Get current molecule
get_molecule_atoms()        # Get atoms from current molecule
get_molecule_bonds()        # Get bonds from current molecule
show_status_message(message, timeout) # Show status message
show_error_message(title, message)    # Show error dialog
show_info_message(title, message)     # Show info dialog
add_menu_item(menu_path, text, callback, shortcut) # Add menu item
get_viewer_2d()                # Get 2D viewer
get_viewer_3d()                # Get 3D viewer
update_viewer_2d()             # Update 2D viewer
update_viewer_3d()             # Update 3D viewer

```

```

# Utility methods
log_info(message)             # Log info message
log_warning(message)          # Log warning message
log_error(message)            # Log error message
log_debug(message)            # Log debug message
...

```

PluginWidget Methods

```

```python
Core widget methods
__init__(plugin) # Initialize widget
get_widget() # Get Qt widget
on_molecule_changed(molecule) # Handle molecule changes
cleanup() # Clean up widget resources
...

```

---

## ## Python API Reference

### ### Core Functions

#### #### Molecular Loading

```

```python
pychem.load(path: str, parallel: bool = True) -> Molecule
    """Load molecular file (PDB, MOL, MOL2, SDF)"""

pychem.parse_smiles(smiles: str) -> Molecule
    """Parse SMILES string into molecule"""
...

```

3D Structure Generation

```

```python

```

```
pychem.generate_3d(mol: Molecule, optimize: bool = True, max_steps: int = 200)
-> None
 """Generate 3D coordinates in-place"""
```

```
pychem.optimize(mol: Molecule, max_iters: int = 500, convergence: float = 1e-
4, method: str = 'lbfgs') -> OptimizationResult
 """Optimize geometry using MMFF94"""
...
```

#### #### Hydrogens and Charges

```
```python
pychem.add_hydrogens(mol: Molecule) -> int
    """Add explicit hydrogens, returns count added"""
```

```
pychem.compute_charges(mol: Molecule) -> None
    """Assign MMFF94 partial charges in-place"""
...
```

Molecular Descriptors

```
```python
pychem.descriptors(mol: Molecule, names: List[str] | None = None) -> dict
 """Calculate molecular descriptors"""

pychem.descriptors_batch(molecules: List[Molecule], names: List[str] | None =
None) -> List[dict]
 """Calculate descriptors for multiple molecules in parallel"""
...
```

#### ### Data Models

##### #### Molecule Class

```
```python
class Molecule:
    # Properties
    atoms: List[Atom]          # List of atoms
    bonds: List[Bond]          # List of bonds
    num_atoms: int              # Number of atoms
    num_bonds: int              # Number of bonds

    # Methods
    molecular_formula() -> str  # Get molecular formula
    add_atom(atom: Atom) -> None # Add atom
    add_bond(bond: Bond) -> None # Add bond
    get_atom(index: int) -> Atom # Get atom by index
    get_bond(index: int) -> Bond # Get bond by index
...
```

Atom Class


```

```python
class Atom:
 # Properties
 symbol: str # Element symbol
 index: int # Atom index
 x, y, z: float # 3D coordinates
 partial_charge: float # Partial charge
 formal_charge: int # Formal charge
 atomic_number: int # Atomic number
 atomic_mass: float # Atomic mass

 # Methods
 distance_to(other: Atom) -> float # Distance to another atom
 is_bonded_to(other: Atom) -> bool # Check if bonded
...

```

#### #### Bond Class

```

```python
class Bond:
    # Properties
    atom1: Atom                # First atom
    atom2: Atom                # Second atom
    order: int                 # Bond order (1, 2, 3)
    is_aromatic: bool          # Aromatic flag
    length: float              # Bond length

    # Methods
    contains_atom(atom: Atom) -> bool # Check if contains atom
    other_atom(atom: Atom) -> Atom    # Get other atom
...

```

Optimization Result

```

```python
class OptimizationResult:
 converged: bool # Did optimization converge?
 final_energy: float # Final energy (kcal/mol)
 num_steps: int # Number of optimization steps
 final_coordinates: ndarray # Final 3D coordinates
...

```

#### ### Service Registry

The ServiceRegistry provides access to all core services:

```

```python
from pychem._bridge import get_registry

registry = get_registry()

```

```

# Access services
forcefield = registry.forcefield      # MMFF94 force field
loader = registry.loader              # File loading service
renderer = registry.renderer          # 3D renderer
descriptors = registry.descriptors    # Descriptor calculator
coordinates = registry.coordinates     # Coordinate generator
'''

```

Event System

PyChem uses an event system for communication:

```

```python
from src.core.events import EventBus

Get event bus
event_bus = EventBus()

Subscribe to events
def on_molecule_changed(event):
 molecule = event.molecule
 print(f"Molecule changed: {molecule.num_atoms} atoms")

event_bus.subscribe('molecule.changed', on_molecule_changed)

Publish events
event_bus.publish('molecule.changed', MoleculeChangedEvent(molecule))
'''

```

---

## ## GUI Features

### ### Main Window Layout

#### #### Menu Bar

- **File:** Open, Save, Export, Print, Exit
- **Edit:** Undo, Redo, Copy, Paste
- **View:** 2D/3D views, Display settings
- **Chemistry:** Optimize, Add hydrogens, Calculate charges, Descriptors
- **Plugins:** Manage plugins, Install plugin
- **Tools:** Python console, Selection tools
- **Help:** About, Documentation

#### #### Toolbar

- Quick access to common functions
- File operations (Open, Save)

- View mode toggles
- Optimization controls
- Selection tools

#### #### Central Widget Area

- **2D Viewer:** Left panel for 2D structure display
- **3D Viewer:** Right panel for 3D molecular visualization
- **Splitter:** Adjustable divider between viewers

#### #### Status Bar

- Current molecule information
- Optimization status
- Selection count
- Performance indicators

### ### 2D Molecular Viewer

#### #### Display Features

- Automatic 2D coordinate generation
- Bond rendering (single, double, triple, aromatic)
- Wedge/hash stereochemical bonds
- Atom labels and formal charges
- Selection highlighting
- Zoom and pan controls

#### #### Interaction

- **Left Click:** Select atom
- **Shift+Click:** Multi-select
- **Shift+Drag:** Rubber band selection
- **Right Click:** Context menu
- **Mouse Wheel:** Zoom
- **Middle+Drag:** Pan

#### #### Rendering Options

- Show/hide atom labels
- Show/hide formal charges
- Bond width adjustment
- Atom size adjustment
- Color schemes (by element, by charge)

### ### 3D Molecular Viewer

#### #### Display Modes

- **Ball and Stick:** Atoms as spheres, bonds as cylinders
- **Space Fill:** Atoms as overlapping spheres
- **Wireframe:** Bonds only, no atoms
- **Cartoon:** Protein secondary structure
- **Ribbon:** Protein backbone trace

- **Backbone:** Only protein backbone atoms

#### #### Color Schemes

- **By Element:** Standard CPK colors
- **By Chain:** Different colors per protein chain
- **By Secondary Structure:** Helix/sheet/coil coloring
- **Rainbow:** Gradient across structure
- **By B-Factor:** Temperature factor coloring
- **By Charge:** Partial charge coloring

#### #### Protein Rendering

- **Alpha Helices:** Cylindrical tubes
- **Beta Sheets:** Flat arrows
- **Loops/Coils:** Thin tubes
- **Cartoon Smoothing:** Catmull-Rom splines
- **Secondary Structure Detection:** DSSP-style algorithm

#### #### Interaction Controls

- **Left Drag:** Rotate molecule
- **Right Drag:** Translate molecule
- **Mouse Wheel:** Zoom in/out
- **Shift+Click:** Select atom
- **Shift+Drag:** Rubber band selection
- **Ctrl+Drag:** Zoom box

#### #### Advanced Features

- **Center of Mass Marker:** Visual COM indicator
- **Centroid Marker:** Geometric center indicator
- **Distance Measurement:** Click two atoms to measure
- **Angle Measurement:** Click three atoms for angle
- **Dihedral Measurement:** Click four atoms for dihedral

#### ### File Operations

##### #### Supported Formats

- **PDB:** Protein Data Bank format
- **MOL:** MDL Molfile format
- **MOL2:** Tripos Mol2 format
- **SDF:** Structure Data File format
- **SMILES:** SMILES string input

##### #### Export Options

- **Image Export:** PNG, SVG, PDF at various DPI (72-600)
- **Ray Traced Images:** High-quality publication images
- **Structure Export:** Save in various molecular formats
- **Print:** Print 2D and 3D views

#### ### Chemistry Operations

#### #### Geometry Optimization

- **MMFF94 Force Field:** Full molecular mechanics (still in developing phase)
- **Hydrogen Addition:** Automatic 3D hydrogen placement
- **Charge Assignment:** BCI partial charge method
- **Optimization Methods:** L-BFGS, steepest descent
- **Convergence Criteria:** Energy and gradient thresholds

#### #### Molecular Properties

- **Descriptor Calculation:** 200+ molecular descriptors
- **Property Categories:** Constitutional, topological, electronic, geometric
- **Batch Processing:** Multiple molecules in parallel
- **Export Results:** CSV, JSON formats

#### ### Python Console

#### #### Features

- **Embedded REPL:** Interactive Python interpreter
- **Molecule Access:** Current molecule as `mol`
- **NumPy Access:** Available as `np`
- **Command History:** Up/down arrow navigation
- **Output Display:** Formatted output and error messages

#### #### Usage Examples

```
```python
# In the console
mol.num_atoms           # Show atom count
mol.molecular_formula() # Show formula
sele('C and within 3.0 of N') # Selection algebra
desc = pychem.descriptors(mol) # Calculate descriptors
```
```

---

#### ## Advanced Usage

#### ### Batch Processing

#### #### Processing Multiple Files

```
```python
import os
import pychem

def process_directory(directory):
    """Process all molecular files in a directory."""
    results = []

    for filename in os.listdir(directory):
```

```

        if filename.endswith(('.pdb', '.mol', '.sdf')):
            filepath = os.path.join(directory, filename)
            try:
                mol = pychem.load(filepath)
                pychem.optimize(mol)
                desc = pychem.descriptors(mol)

                results.append({
                    'filename': filename,
                    'molecule': mol,
                    'descriptors': desc
                })
            except Exception as e:
                print(f"Error processing {filename}: {e}")

    return results

# Usage
results = process_directory('molecules/')
for result in results:
    print(f"{result['filename']}: MW = {result['descriptors']['molecular_weight']:.2f}")
'''

#### Parallel Descriptor Calculation
```python
import pychem

Large dataset processing
smiles_list = [...] # Large list of SMILES strings
molecules = [pychem.parse_smiles(s) for s in smiles_list]

Batch processing with parallel execution
descriptors = pychem.descriptors_batch(molecules)

Analyze results
weights = [d['molecular_weight'] for d in descriptors]
print(f"Average molecular weight: {sum(weights)/len(weights):.2f}")
'''

Custom Analysis Workflows

Property Correlation Analysis
```python
import numpy as np
import pychem
from scipy import stats

```

```

def analyze_property_correlation(smiles_list, property_values):
    """Analyze correlation between descriptors and properties."""

    # Generate molecules and descriptors
    molecules = [pychem.parse_smiles(s) for s in smiles_list]
    descriptors = pychem.descriptors_batch(molecules)

    # Extract specific descriptors
    weights = [d['molecular_weight'] for d in descriptors]
    logp_values = [d['logp'] for d in descriptors if 'logp' in d]

    # Calculate correlations
    correlations = {}
    for i, desc in enumerate(descriptors):
        for desc_name, desc_value in desc.items():
            if isinstance(desc_value, (int, float)):
                corr, p_value = stats.pearsonr([property_values[i]],
[desc_value])
                correlations[desc_name] = (corr, p_value)

    return correlations

# Usage
smiles = ["CCO", "c1ccccc1", "CC(=O)O"]
properties = [0.5, 1.2, 0.8] # Example property values
correlations = analyze_property_correlation(smiles, properties)

```

Conformer Analysis

```

```python
def generate_conformer_ensemble(smiles, n_conformers=50):
 """Generate multiple conformers and analyze ensemble."""

 mol = pychem.parse_smiles(smiles)
 pychem.generate_3d(mol, optimize=False) # Initial coordinates

 conformers = []
 energies = []

 for i in range(n_conformers):
 # Copy molecule and randomize coordinates slightly
 conformer = copy.deepcopy(mol)

 # Small random perturbation
 for atom in conformer.atoms:
 atom.x += np.random.normal(0, 0.1)
 atom.y += np.random.normal(0, 0.1)
 atom.z += np.random.normal(0, 0.1)

```

```

 # Optimize
 result = pychem.optimize(conformer, max_iters=100)
 conformers.append(conformer)
 energies.append(result.final_energy)

 # Sort by energy
 sorted_indices = np.argsort(energies)
 return [conformers[i] for i in sorted_indices], [energies[i] for i in
sorted_indices]
'''

```

### ### Integration with Other Tools

#### #### Jupyter Notebook Integration

```

'''python
In Jupyter notebook
import pychem
import matplotlib.pyplot as plt
%matplotlib inline

Load and analyze molecules
mol = pychem.parse_smiles("CCO")
pychem.optimize(mol)

Plot molecular properties
desc = pychem.descriptors(mol)
plt.bar(desc.keys(), desc.values())
plt.xticks(rotation=45)
plt.title("Molecular Descriptors")
plt.show()
'''

```

#### #### Data Export and Analysis

```

'''python
import pandas as pd
import pychem

def create_descriptor_dataset(smiles_list):
 """Create a pandas DataFrame with molecular descriptors."""

 molecules = [pychem.parse_smiles(s) for s in smiles_list]
 descriptors = pychem.descriptors_batch(molecules)

 # Create DataFrame
 df = pd.DataFrame(descriptors)
 df['smiles'] = smiles_list

```



```

 return df

Usage
smiles = ["CCO", "c1ccccc1", "CC(=O)O", "CCCCC"]
df = create_descriptor_dataset(smiles)
print(df.head())

Save to CSV
df.to_csv('molecular_descriptors.csv', index=False)
'''

Performance Optimization

Memory Management
```python
# For large datasets, process in chunks
def process_large_dataset(smiles_list, chunk_size=100):
    """Process large datasets in chunks to manage memory."""

    results = []
    for i in range(0, len(smiles_list), chunk_size):
        chunk = smiles_list[i:i+chunk_size]
        molecules = [pychem.parse_smiles(s) for s in chunk]
        chunk_results = pychem.descriptors_batch(molecules)
        results.extend(chunk_results)

        # Clear memory
        del molecules
        del chunk_results

    return results
'''

#### Parallel Processing
```python
Use parallel processing for CPU-intensive tasks
from concurrent.futures import ProcessPoolExecutor

def optimize_molecule(smiles):
 """Optimize a single molecule."""
 mol = pychem.parse_smiles(smiles)
 pychem.optimize(mol)
 return mol

def parallel_optimize(smiles_list):
 """Optimize multiple molecules in parallel."""

 with ProcessPoolExecutor() as executor:

```

```
 molecules = list(executor.map(optimize_molecule, smiles_list))

 return molecules

```

## **## Troubleshooting**

### **### Common Installation Issues**

#### **#### Python Version Compatibility**

```
```bash
# Check Python version
python --version

# If version < 3.10, upgrade Python
# PyChem requires Python 3.10 or newer
```
```

#### **#### Qt/PySide6 Issues**

```
```bash
# If PySide6 installation fails
pip install --upgrade pip
pip install --no-cache-dir PySide6

# For macOS with M1/M2 chips
pip install PySide6 --extra-index-url https://pypi.org/simple
```
```

#### **#### Virtual Environment Issues**

```
```bash
# If activation fails on Windows PowerShell
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
venv\Scripts\Activate.ps1

# If modules not found after activation
pip install -r requirements.txt --force-reinstall
```
```

### **### Common Runtime Issues**

#### **#### Molecule Loading Errors**

```
```python
# Check file format support
try:
    mol = pychem.load("molecule.pdb")
except Exception as e:
```

```

    print(f"Error loading file: {e}")
    # Check if file exists and is readable
    import os
    print(f"File exists: {os.path.exists('molecule.pdb')}")
    ...

#### Memory Issues with Large Molecules
```python
For large proteins, use parallel loading
mol = pychem.load("large_protein.pdb", parallel=True)

Monitor memory usage
import psutil
process = psutil.Process()
print(f"Memory usage: {process.memory_info().rss / 1024 / 1024:.1f} MB")
...

Optimization Convergence Issues
```python
# If optimization fails to converge
result = pychem.optimize(mol, max_iters=1000, convergence=1e-6)

if not result.converged:
    print("Optimization did not converge")
    print(f"Final energy: {result.final_energy}")
    print(f"Number of steps: {result.num_steps}")

    # Try different method
    result = pychem.optimize(mol, method='steepest_descent')
...

### Performance Issues

#### Slow 3D Rendering
```python
For large molecules, reduce visual quality
Use wireframe mode instead of ball-and-stick
Hide hydrogen atoms for large proteins

Enable parallel processing
mol = pychem.load("large_mol.pdb", parallel=True)
...

Slow Descriptor Calculation
```python
# Use batch processing for multiple molecules
molecules = [pychem.parse_smiles(s) for s in smiles_list]
descriptors = pychem.descriptors_batch(molecules) # Parallel execution

```

```
# Or calculate only needed descriptors
specific_desc = pychem.descriptors(mol, names=['molecular_weight', 'logp'])
...
```

Plugin Issues

Plugin Loading Failures

```
```python
Check plugin dependencies
from src.plugins.plugin_manager import PluginManager

manager = PluginManager()
plugins = manager.discover_plugins()

for plugin in plugins:
 print(f"Plugin: {plugin.info.name}")
 print(f"Dependencies: {plugin.info.dependencies}")

 # Check if dependencies are available
 for dep in plugin.info.dependencies:
 try:
 __import__(dep)
 print(f" {dep}: OK")
 except ImportError:
 print(f" {dep}: MISSING")
...

```

#### #### Plugin Development Debugging

```
```python
# Enable plugin logging
import logging
logging.basicConfig(level=logging.DEBUG)

# Plugin will log debug information
plugin_logger = logging.getLogger("plugin")
plugin_logger.setLevel(logging.DEBUG)
...

```

File Format Issues

PDB File Problems

```
```python
Check PDB file format
def validate_pdb_file(filepath):
 """Basic PDB file validation."""

 with open(filepath, 'r') as f:

```

```

 lines = f.readlines()

 atom_lines = [line for line in lines if line.startswith('ATOM') or
line.startswith('HETATM')]

 if not atom_lines:
 print("No ATOM/HETATM records found")
 return False

 # Check coordinate format
 for line in atom_lines[:5]: # Check first 5 lines
 try:
 x = float(line[30:38].strip())
 y = float(line[38:46].strip())
 z = float(line[46:54].strip())
 except ValueError:
 print(f"Invalid coordinates in line: {line}")
 return False

 print(f"PDB file appears valid: {len(atom_lines)} atoms")
 return True
 ...

```

### ### Getting Help

#### #### Debug Information

```

```python
# Get system information for bug reports
import sys
import pychem
import numpy as np
from PySide6 import QtCore

print("PyChem Debug Information:")
print(f"Python version: {sys.version}")
print(f"PyChem version: {pychem.__version__}")
print(f"NumPy version: {np.__version__}")
print(f"PySide6 version: {QtCore.__version__}")
print(f"Platform: {sys.platform}")
```

```

#### #### Error Reporting

When reporting issues, include:

1. Operating system and version
2. Python and PyChem versions
3. Complete error traceback
4. Minimal reproducer (SMILES string or file)
5. Steps to reproduce the issue

---

## **## Development Guide**

### **### Code Structure**

#### **#### Core Modules**

- `src/core/domain/`: Core data models (Molecule, Atom, Bond)
- `src/core/protocols/`: Service interfaces (IForceField, IRenderer, etc.)
- `src/services/`: Service implementations
- `src/features/`: Feature modules (visualization, parsing, etc.)
- `src/app/`: GUI application layer
- `plugins/`: Plugin implementations

#### **#### Service Architecture**

PyChem uses a service-oriented architecture with protocol interfaces:

```
```python
# Service protocol
from src.core.protocols import IForceField

class MyForceField:
    def add_hydrogens(self, mol): ...
    def optimize_geometry(self, mol): ...
    def compute_energy(self, mol): ...

# Register service
from src.core.registry import ServiceRegistry
registry = ServiceRegistry()
registry.forcefield = MyForceField()
```
```

### **### Contributing Guidelines**

#### **#### Code Style**

- Follow PEP 8 with 4-space indentation
- Maximum 100 characters per line
- Use type hints everywhere reasonable
- Document all public functions and classes

#### **#### Testing**

```
```bash
# Run all tests
for t in tests/test_*.py; do python3 "$t"; done

# Run specific test
python3 tests/test_mmff94_service.py
```
```

...

#### #### Submitting Changes

1. Fork the repository
2. Create a feature branch
3. Make changes with tests
4. Run full test suite
5. Submit pull request

### ### Plugin Development

#### #### Creating New Plugins

1. Inherit from `BasePlugin`
2. Implement required methods
3. Create widget class
4. Add to plugins directory
5. Test with plugin manager

#### #### Plugin Distribution

- Share plugin files directly
- Create plugin packages
- Document dependencies
- Provide usage examples

### ### API Extensions

#### #### Adding New Services

1. Define protocol interface
2. Implement service class
3. Register in ServiceRegistry
4. Update public API if needed
5. Add tests and documentation

#### #### Extending Molecular Models

1. Modify core domain classes
2. Update serialization
3. Modify all services
4. Add tests for new features
5. Update documentation

---

## ## Appendices

### ### Appendix A: SMILES Syntax Reference

#### #### Basic Syntax

- `C` - Carbon atom

- `O` - Oxygen atom
- `N` - Nitrogen atom
- `=` - Double bond
- `#` - Triple bond
- `:` - Aromatic bond

#### #### Advanced Features

- `[ ]` - Bracketed atoms (charges, isotopes)
- `@` - Stereochemistry
- `()` - Branching
- `1-9` - Ring closures

### ### Appendix B: File Format Support

#### #### PDB Format

- ATOM records
- HETATM records
- CONECT connectivity
- SEQRES sequences

#### #### MOL Format

- Header block
- Connection table
- Charge information

#### #### SDF Format

- Multiple molecules
- Property data
- 2D/3D coordinates

### ### Appendix C: Descriptor Categories

#### #### Constitutional

- Molecular weight
- Atom counts
- Bond counts
- Ring counts

#### #### Topological

- Wiener index
- Randic index
- Balaban index
- Connectivity indices

#### #### Electronic

- Dipole moment
- HOMO/LUMO gap
- Electronegativity



- Hardness/softness

#### #### Geometric

- Surface area
- Volume
- Shape indices
- Moment of inertia

### ### Appendix D: Performance Benchmarks

#### #### Typical Performance Metrics

- Small molecule optimization: ~0.3 seconds
- Protein (1000 atoms): ~5 seconds
- Descriptor calculation (100 molecules): ~2 seconds
- 3D rendering (500 atoms): ~60 FPS

#### #### Scaling Behavior

- Optimization:  $O(n^2)$  for pairwise interactions
- Rendering:  $O(n)$  with culling optimizations
- Descriptors:  $O(n)$  for most calculations
- File loading:  $O(n)$  with parallel processing

---

## ## License and Citation

### ### License

PyChem is released under the MIT License. See LICENSE file for details.

### ### Citation

If you use PyChem in academic work, please cite:

...

```
@software{pychem,
 title = {PyChem: A Pure-Python Cheminformatics and Molecular Visualization
Toolkit},
 author = {Masand, Gaurav and Masand, Vijay},
 year = {2026},
 url = {https://github.com/vijaymasand/PyChem},
 note = {Accessed: YYYY-MM-DD}
}
```

...

### ### Acknowledgments

- MMFF94 force field: Thomas A. Halgren (Merck, 1996)
- OASA library: Beda Kosata
- PySide6/Qt: Qt Project
- NumPy: NumPy development team

---

## **## Contact and Support**

### **### GitHub Repository**

- <https://github.com/vijaymasand/PyChem>

### **### Issue Reporting**

- <https://github.com/vijaymasand/PyChem/issues>

### **### Documentation**

- <https://github.com/vijaymasand/PyChem/docs>

### **### Community**

- Discussions: [GitHub Discussions](#)
- Issues: [GitHub Issues](#)
- Wiki: [GitHub Wiki](#)

---

*\*This manual covers PyChem version 2.0.0. For the latest updates, visit the GitHub repository.\**